FACULDADE DE TECNOLOGIA SANTANA DE PARNAÍBA
CURSO SUPERIOR DE TECNOLOGIA EM
ANÁLISE E DESENVOLVIMENTO DE SISTEMAS (ADS)

PLENNA: IMPLEMENTAÇÃO E VALIDAÇÃO DE UM SERVIDOR MCP

ATIVIDADE PRÁTICA — 6º SEMESTRE

MARINA CRUZ DE SOUZA
SANDY PEREIRA CAVALCANTI

Santana de Parnaíba, SP
2026

1 INTRODUÇÃO

A integração de modelos de inteligência artificial (IA) com dados externos exige mecanismos que permitam distinguir uma resposta gerada pelo modelo de uma operação executada por um sistema. O Model Context Protocol (MCP) oferece uma forma padronizada de disponibilizar ferramentas, recursos e orientações a clientes compatíveis. Nesta atividade, o objetivo foi construir um servidor funcional, testá-lo independentemente de uma IA e, posteriormente, demonstrar seu uso por um modelo de linguagem.

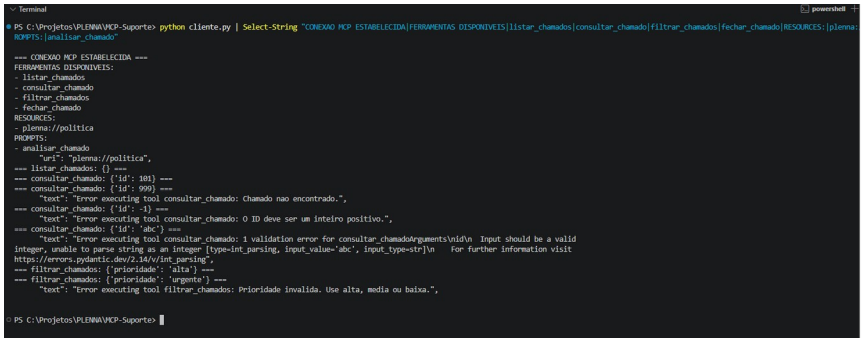
Como contexto, adotou-se o PLENNA, projeto de aplicativo móvel voltado à reflexão sobre hábitos de consumo digital. Foi desenvolvido um módulo de suporte separado da aplicação principal, com quatro chamados fictícios relacionados a login, gráficos, metas e histórico de compras. Nenhuma informação real de usuários, API ou banco de dados do aplicativo foi acessada.

2 IMPLEMENTAÇÃO E FLUXO DE EXECUÇÃO

O servidor foi implementado em Python com FastMCP e transporte stdio. O arquivo servidor.py publica quatro ferramentas: listar_chamados, consultar_chamado, filtrar_chamados e fechar_chamado; uma Resource, plenna://politica; e o Prompt analisar_chamado. Os registros são mantidos em dados/chamados.json. Os clientes cliente.py e cliente_operacao.py permitem verificar as funcionalidades diretamente pelo protocolo.

O fluxo de consulta é iniciado pelo cliente MCP, que envia o nome da ferramenta e seus argumentos. O servidor valida a entrada, consulta o JSON e devolve o resultado. Na escrita, a ferramenta fechar_chamado altera o status e persiste a mudança. O cliente_operacao.py solicita confirmação explícita antes da chamada, mas essa proteção não é imposta universalmente a outros hosts. As operações relevantes são registradas em operacoes.log.

Figura 1 – Descoberta das ferramentas, Resource e Prompt pelo cliente Python.



Fonte: Elaboração própria (2026).

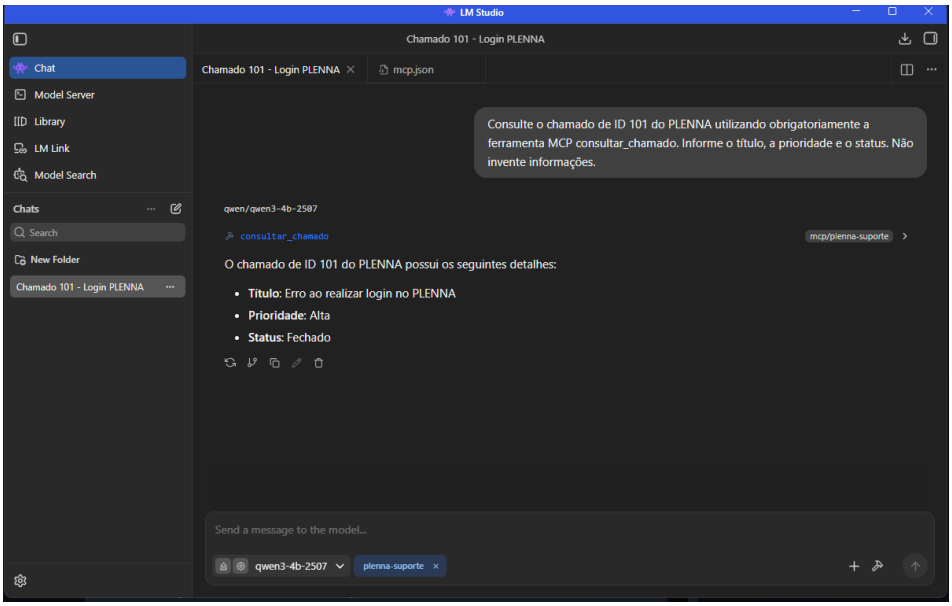
3 TESTES REALIZADOS E RESULTADOS

No cliente Python, foram testadas a listagem dos quatro chamados, a consulta do ID 101, o erro para o ID 999, a rejeição do ID negativo e de texto não numérico, a filtragem por prioridade alta e a rejeição da prioridade urgente. Também foram verificadas a recusa de fechamento pelo usuário, a confirmação da operação, a persistência da alteração e a resposta ao tentar fechar um chamado já encerrado.

Nos testes de escrita, o chamado 102 foi fechado mediante confirmação em uma execução inicial. Em uma nova rodada documentada por capturas do terminal, o cliente cancelou o fechamento do chamado 103 quando o usuário respondeu “nao” e o executou somente após a confirmação “sim”. O log registrou o fechamento do ID 103 às 19h20min28s, e o JSON foi consultado para conferir a persistência.

Para a etapa de IA, foi utilizado o LM Studio com o modelo local Qwen3 4B Instruct 2507. O servidor foi cadastrado em mcp.json e habilitado no chat. O modelo solicitou a ferramenta consultar_chamado para o ID 101; após autorização, a resposta apresentou o título “Erro ao realizar login no PLENNA”, prioridade alta e status fechado. A Figura 2 documenta o resultado exibido pelo host.

Figura 2 – Resultado da consulta do chamado 101 pelo Qwen3 via MCP.

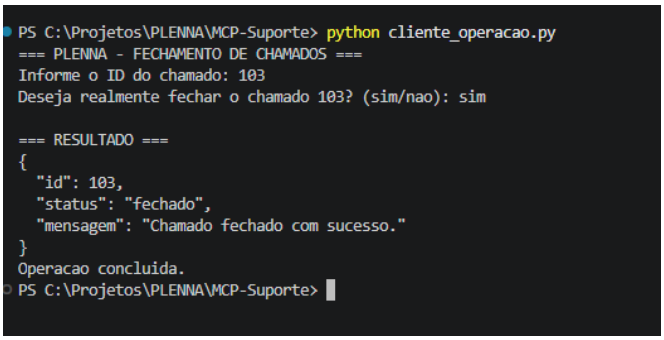


Fonte: Elaboração própria (2026).

3 TESTES REALIZADOS E RESULTADOS (CONTINUAÇÃO)

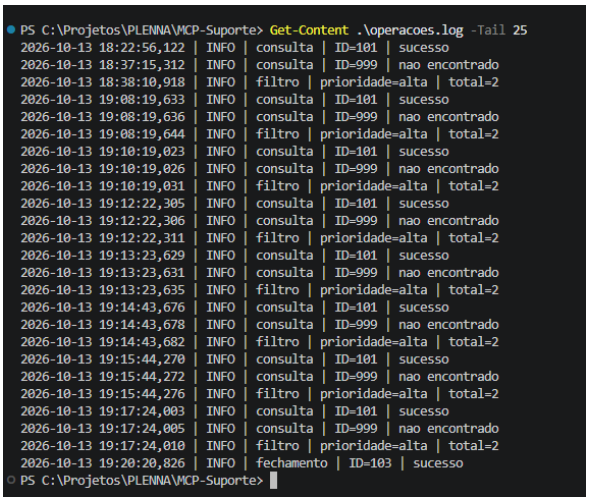
Na integração com IA, a consulta do ID 999 retornou ausência do registro. A filtragem por prioridade alta retornou os chamados 101 e 104. As chamadas foram verificadas no arquivo operacoes.log. Separadamente, os testes locais comprovaram a recusa de escrita, a confirmação do fechamento do ID 103 e o registro dessa operação no servidor, conforme as Figuras 3 e 4.

Figura 3 – Fechamento do chamado 103 após confirmação do usuário.



Fonte: Elaboração própria (2026).

Figura 4 – Log de operações, incluindo o fechamento bem-sucedido do chamado 103.



Fonte: Elaboração própria (2026).

4 SEGURANÇA E CONSIDERAÇÕES FINAIS

O experimento foi restrito a dados fictícios e execução local. Embora o cliente de fechamento exija confirmação, a ferramenta de escrita pode ser chamada por outros clientes MCP; por isso, um sistema real precisaria aplicar autorização também no servidor. O armazenamento em JSON não oferece transações, controle de concorrência ou auditoria imutável. A Resource de política orienta o uso, mas não substitui mecanismos técnicos de segurança.

Os resultados demonstraram a descoberta e a execução de ferramentas MCP, a validação de entradas, a persistência de uma alteração e a integração com IA sem contratação de API externa. A principal conclusão é que a resposta do modelo somente se torna verificável quando associada à chamada de ferramenta e ao registro da operação no servidor. Os arquivos de evidências e o README permitem reproduzir e conferir os testes.